

Trabalho Prático 2

Memória virtual

Davi Fraga Marques Neves

Matrícula: 2020420575

Vinicius Trindade Dias Abel

Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte – MG – Brasil

davifmn@gmail.com, viniciustda@ufmg.br

1. Introdução

1.1. Memória virtual

A Memória Virtual é uma técnica essencial utilizada pelos sistemas operacionais modernos para gerenciar e otimizar o uso da memória principal (RAM). Essa abordagem oferece aos programas a ilusão de dispor de uma grande quantidade de memória contínua, mesmo que a capacidade física da RAM seja limitada. O mecanismo funciona criando um espaço de endereçamento lógico que é mapeado tanto para a memória física quanto para áreas do disco rígido, conhecidas como swap.

Entre as principais vantagens da memória virtual, destaca-se a possibilidade de executar programas maiores do que a capacidade física da RAM, graças ao armazenamento temporário de dados no disco rígido. Além disso, a técnica garante isolamento entre processos, evitando que falhas em um programa afetem a memória de outro, o que aumenta a segurança e a estabilidade do sistema. Outra vantagem é a eficiência na multiplexação da memória, permitindo a execução simultânea de múltiplos programas de maneira mais fluida. Por fim, a memória virtual simplifica o desenvolvimento de software, pois os programadores podem trabalhar com um espaço de endereçamento lógico contínuo, sem se preocupar com fragmentação ou limitações físicas da memória.

1.2. Objetivo

O objetivo deste trabalho é simular o funcionamento de uma memória virtual, replicando suas estruturas e o gerenciamento de memória, para ilustrar os conceitos abordados em sala de aula sobre algoritmos de substituição de páginas e o mapeamento da memória virtual. Para isso, desenvolvemos um programa em linguagem C que recebe, como entrada, um arquivo contendo a sequência de endereços de memória acessados por um programa real, classificados como operações de escrita (W) ou leitura (R).

Durante a execução, definimos o tamanho dos quadros de página e selecionamos um algoritmo de substituição de páginas. Dessa forma, buscamos compreender o comportamento de cada algoritmo de substituição em diferentes modelos de tabelas de páginas.

2. Decisões de projeto

Para este projeto, adotamos uma abordagem metódica e detalhada para garantir seu pleno funcionamento. Desde a elaboração de uma visão panorâmica sobre a divisão das estruturas do código até a implementação final, seguimos um planejamento cuidadoso.

O primeiro passo foi a leitura do arquivo de entrada, garantindo que os argumentos fornecidos fossem válidos e estivessem na posição correta. Em seguida, criamos estruturas de dados para armazenar as informações necessárias e modelar o comportamento de cada algoritmo e tabela de páginas.

Na etapa inicial de implementação, desenvolvemos os métodos relacionados à inicialização das tabelas de páginas, contemplando os modelos normal, invertida e hierárquicas de 2 e 3 níveis. Além disso, implementamos cinco algoritmos de substituição de páginas, aplicáveis a cada um desses tipos de tabela.

Posteriormente, criamos métodos específicos para cada uma das quatro formas de tabela. Esses métodos permitem escolher, com base nos dados fornecidos na entrada, qual algoritmo de substituição será executado em combinação com a tabela selecionada.

2.1. Estrutura de Dados

As instruções do projeto recomendavam o uso de structs para organizar o código, e seguimos essa orientação. Com os dados lidos, criamos uma struct principal chamada `estadoSimulacao`, utilizada em todos os métodos para armazenar o estado atual da memória virtual durante a simulação.

Além disso, definimos outras duas structs fundamentais para o projeto: página e quadro. Elas representam os elementos centrais para a simulação da memória virtual e física, respectivamente.

Essa abordagem facilitou o mapeamento do que acontecia em cada método durante as simulações, tornando o código mais organizado e permitindo uma maior reutilização das estruturas e lógica implementadas.

2.2. Métodos

Um método inicial e extremamente necessário neste código é o `iniciaEstadoSimulacao`, responsável por inicializar todas as variáveis, estados e quadros utilizados nas simulações subsequentes. Essa etapa é essencial para garantir que o ambiente de simulação esteja preparado antes do início de cada execução.

Embora essa inicialização pudesse ter sido realizada diretamente na declaração das structs (uma escolha pessoal que, na prática, não faz muita diferença), optamos por centralizá-la no método, garantindo maior clareza e organização. O objetivo principal desse método é configurar corretamente todas as variáveis necessárias antes do processamento da simulação.

```
typedef struct estadoSimulacao {
    quadro *memoria;           // Vetor de quadros representando a
                                // memória física.
    int tam_memoria;           // Número total de quadros
                                // disponíveis na memória.
    int atual_tam_memoria;      // Quantidade de quadros atualmente
                                // em uso.

    pagina *tabela_paginas;    // Tabela de páginas convencional.
    int tam_tabela_paginas;    // Tamanho total da tabela de
                                // páginas.

    lista *lista_id;           // Lista encadeada para algoritmos
                                // baseados em ordenação.
    lista *lista_circular;     // Lista circular para algoritmos
                                // como Segunda Chance.

    invertida *tabela_paginas_invertida; // Tabela de páginas
                                // invertida.

    pagina **tabela_pagina_n2; // Representação de uma tabela
                                // hierárquica de 2 níveis.
    pagina ***tabela_pagina_n3; // Representação de uma tabela
                                // hierárquica de 3 níveis.

    int bits_n1;               // Número de bits do primeiro nível
                                // em tabelas hierárquicas.
    int bits_n2;               // Número de bits do segundo nível.
    int bits_n3;               // Número de bits do terceiro
                                // nível.
}
```

```

    int num_bits_endereco;    // Número total de bits do endereço
virtual.

    int tam_total_mem;        // Tamanho total da memória física
(em KB).

    int pageFaults;          // Contador de Page Faults.
    int acessos_disco;        // Contador de acessos ao disco.
    int acessos_memoria;      // Contador de acessos à memória.
    int acessos_mem_R;        // Contador de acessos de leitura à
memória.
    int acessos_mem_W;        // Contador de acessos de escrita à
memória.
    int paginas_sujas;        // Contador de páginas que foram
modificadas.
} estadoSimulacao;

```

Em seguida criamos métodos para inicializar todas as tabelas:

```

void inicializaTabelaPaginasInvertida(estadoSimulacao *estado);
void inicializaN1TabelaN2(estadoSimulacao *estado);
void inicializaN2TabelaN2(estadoSimulacao *estado, int id);
void inicializaN1TabelaN3(estadoSimulacao *estado);
void inicializaN2TabelaN3(estadoSimulacao *estado, int n1_id);
void inicializaN3TabelaN3(estadoSimulacao *estado, int n1_id, int
n2_id);
void inicializaMemoria(estadoSimulacao *estado);
void inicializaListaAux(lista **lista){
    *lista = malloc(sizeof(lista));
    (*lista)->tamanho = 0;
    (*lista)->primeiro = NULL;
    (*lista)->ultimo = NULL;
}

```

Substituição de páginas

Cada combinação entre algoritmo de substituição de página e cada tipo de paginação recebeu um método próprio:

```

void densaFIFO(estadoSimulacao *estado, int endereco);
void densaRandom(estadoSimulacao *estado, int endereco);
void densaSegundaChance(estadoSimulacao *estado, int endereco);
void densaLRU(estadoSimulacao *estado, int endereco);

```

```

int invertidaFIFO(estadoSimulacao *estado, int endereco);
int invertidaRandom(estadoSimulacao *estado, int endereco);
int invertidaSegundaChance(estadoSimulacao *estado, int endereco);
int invertidaLRU(estadoSimulacao *estado, int endereco);
int buscaTabelaInvertida(estadoSimulacao *estado, int
endereco_tabela);
void hierarquica2FIFO(estadoSimulacao *estado, int endereco, int
offset);
void hierarquica2Random(estadoSimulacao *estado, int endereco, int
offset);
void hierarquica2SegundaChance(estadoSimulacao *estado, int
endereco, int offset);
void hierarquica2LRU(estadoSimulacao *estado, int endereco, int
offset);
void hierarquica3FIFO(estadoSimulacao *estado, int endereco, int
offset);
void hierarquica3Random(estadoSimulacao *estado, int endereco, int
offset);
void hierarquica3SegundaChance(estadoSimulacao *estado, int
endereco, int offset);
void hierarquica3LRU(estadoSimulacao *estado, int endereco, int
offset);

```

Tabelas Hierárquicas

As tabelas hierárquicas precisam ter os endereços calculador baseados no deslocamento do offset para calcular o próximo nível.

```

int getIdNivel2Tabela1(unsigned endereco, int offset, int bits_n1,
int bits_n2);
int getIdNivel2Tabela2(unsigned endereco, int offset, int
bits_n2);
int getIdNivel3Tabela1(unsigned endereco, int offset, int bits_n1,
int bits_n2, int bits_n3);
int getIdNivel3Tabela2(unsigned endereco, int offset, int bits_n2,
int bits_n3);
int getIdNivel3Tabela3(unsigned endereco, int offset, int
bits_n3);

```

Menção honrosa

Funções auxiliares como determinar o offset e a validade da leitura de dados.

A simulação

Com as memórias definidas, inicializadas e organizadas, é finalmente possível iniciar a simulação. Neste ponto, dispomos de tudo o que é necessário para começar a simular a memória virtual.

Todos os métodos de simulação recebem uma instância da struct estadoSimulacao, o arquivo de entrada lido, o tamanho do offset e o algoritmo que será utilizado. Dessa forma, as funções de cada algoritmo são chamadas e iniciadas para cada versão de atualização de tabela definida durante a leitura dos argumentos, especificando também o tipo de tabela em que cada algoritmo será executado.

```
void memoriaVirtual(estadoSimulacao *estado, FILE *arquivo, int
offset, char *algoritmo);
void memoriaVirtualInvertida(estadoSimulacao *estado, FILE
*arquivo, int offset, char *algoritmo);
void memoriaVirtualHierarquica2(estadoSimulacao *estado, FILE
*arquivo, int offset, char *algoritmo);
void memoriaVirtualHierarquica3(estadoSimulacao *estado, FILE
*file, int offset, char *algoritmo);
```

2.3. Configuração para teste

- Sistema Operacional do computador: Linux Ubuntu;
- Linguagem de programação implementada: C;
- Compilador utilizado: GCC;
- Dados do seu processador: i7;
- Quantidade de memória RAM: 16GB.

3. Dados coletados

```
=====
Executando o simulador...

===== Relatorio de Execucao =====

Arquivo de entrada: tp2-files/compilador.log
Tamanho da memoria: 128 KB
Tamanho das paginas: 4 KB
Tecnica de reposicao: lru
Tipo de tabela: densa
Numero total de acessos a memoria: 1000000
Numero de page faults: 84401
Numero total de acessos de leitura: 892816
Numero total de acessos de escrita: 107184
Numero total de paginas sujas: 11737

=====
```

```
=====
Executando o simulador...

===== Relatorio de Execucao =====

Arquivo de entrada: tp2-files/compilador.log
Tamanho da memoria: 128 KB
Tamanho das paginas: 4 KB
Tecnica de reposicao: lru
Tipo de tabela: hierarquica2
Numero total de acessos a memoria: 1000000
Numero de page faults: 66936
Numero total de acessos de leitura: 892816
Numero total de acessos de escrita: 107184
Numero total de paginas sujas: 10436

=====
```

4. Conclusões

Este trabalho teve como objetivo desenvolver um simulador de memória virtual, permitindo uma aplicação prática dos conceitos abordados em sala de aula sobre gerenciamento de memória. A implementação incluiu a replicação das

estruturas de um sistema de memória virtual e a simulação de diferentes algoritmos de substituição de páginas em múltiplos modelos de tabelas: densa, invertida e hierárquicas de 2 e 3 níveis.

Por meio desse projeto, foi possível explorar os impactos das diferentes estratégias de substituição de páginas, compreendendo como o comportamento da memória virtual varia em função do algoritmo e do modelo de tabela utilizado. A criação de funções para inicialização, manipulação e atualização das estruturas de dados foi um desafio importante, exigindo decisões criteriosas para equilibrar desempenho e organização do código.

Entre os principais aprendizados, destacam-se a análise do comportamento de page faults em relação ao tamanho da memória e das páginas, bem como o impacto do uso de tabelas hierárquicas e invertidas no consumo de memória e na eficiência das operações. A comparação prática dos algoritmos FIFO, LRU, Random e Segunda Chance também evidenciou as vantagens e desvantagens de cada abordagem em diferentes cenários.

Assim, o trabalho proporcionou uma experiência rica no uso de estruturas de dados, no desenvolvimento de simuladores e na análise de desempenho de algoritmos, consolidando os conhecimentos sobre sistemas operacionais e memória virtual.

5. Instruções para compilação e execução

- Acesse o diretório tp2virtual
- Utilizando o terminal, compile o TP 02 utilizando o comando: make
- Com esse comando, será gerado o arquivo tp2virtual.exe e os arquivos *.o
- Utilizando o terminal, teste o tp2virtual.exe passando os argumentos pela linha de comando: tp2virtual <algoritmo> <arquivo_entrada> <tamanho_pagina> <tamanho_memoria> [tabela]

Obs.: [tabela] é opcional. Você pode passar os parâmetros: densa, hierarquica2, hierarquica3, inversa. Caso não seja passado argumento, será utilizada a tabela densa

- Para remover os arquivos objeto (.o) e o executável: make clean
- Para remover apenas os arquivos objeto (.o): make clean-obj

